

Глубинное обучение

Лекция 8

Перенос обучения. Распознавание лиц.

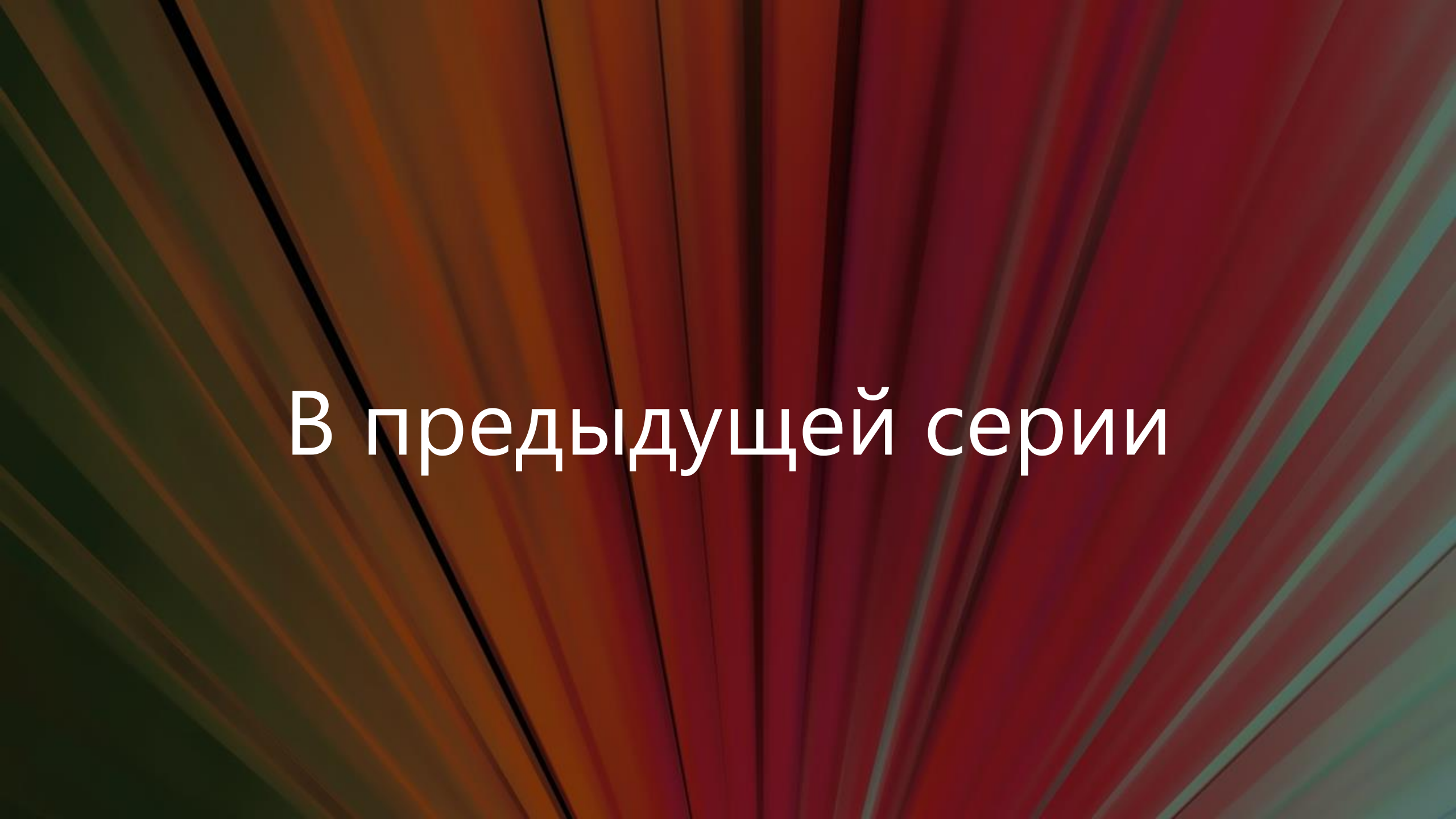
Михаил Гущин

mhushchyn@hse.ru

НИУ ВШЭ, 2025



НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ



В предыдущей серии

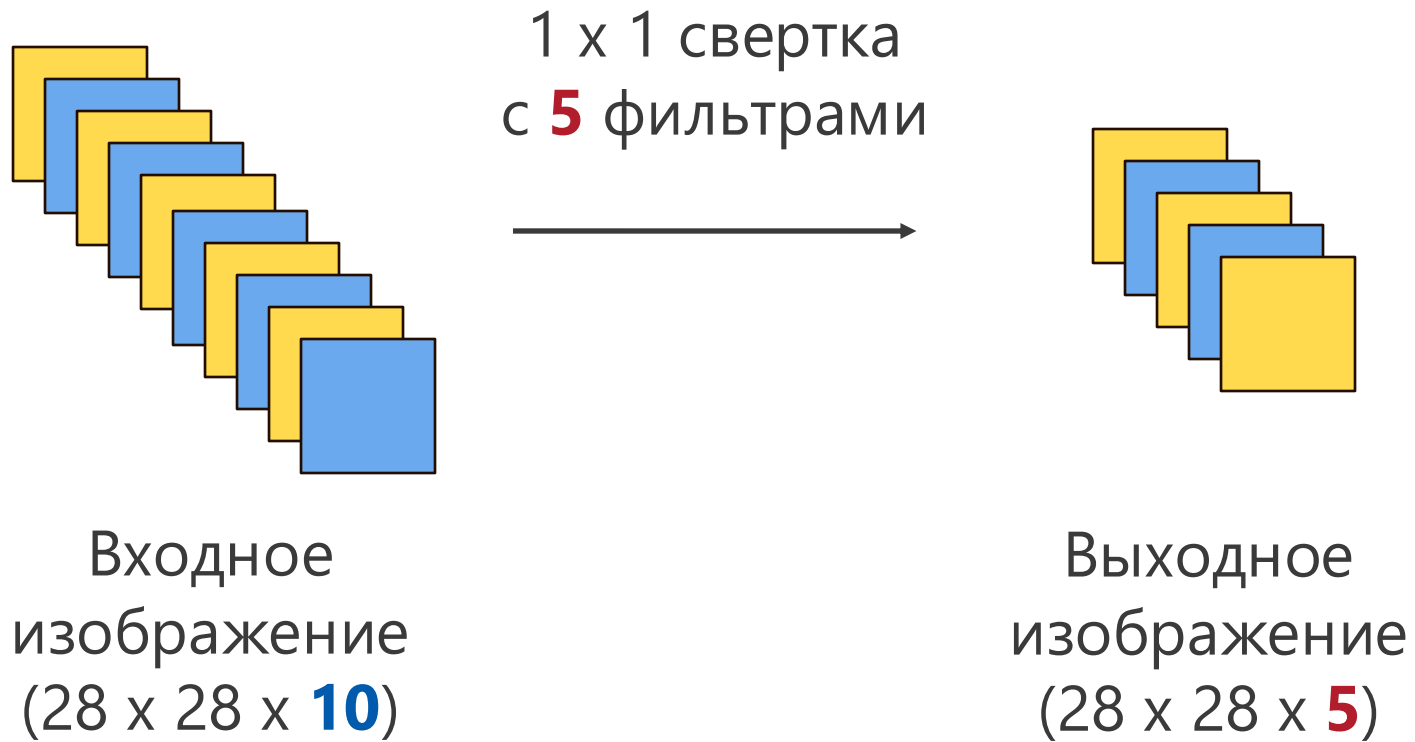
ImageNet



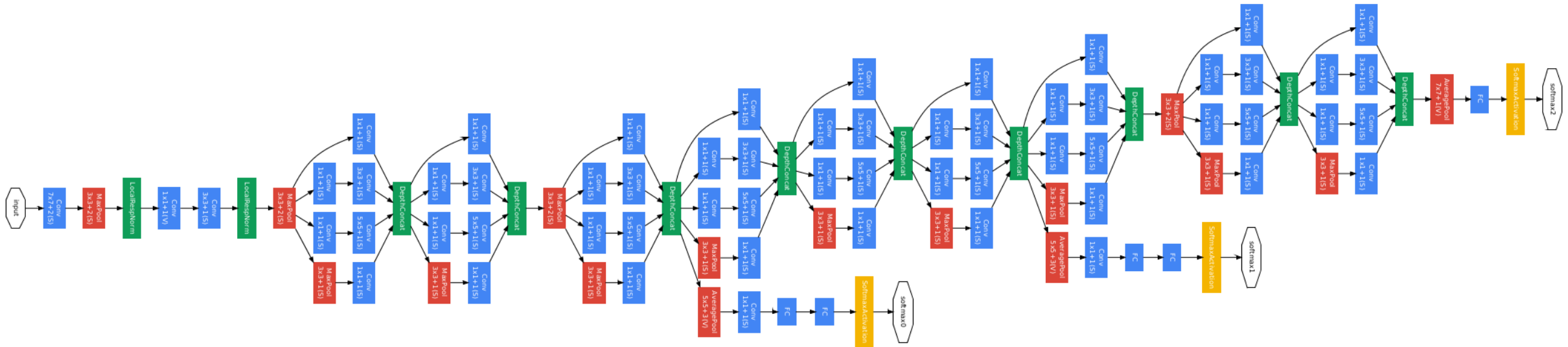
Подробнее: <https://image-net.org/challenges/LSVRC>

- ▶ ImageNet Large Scale Visual Recognition Challenge (ILSVRC)
- ▶ 1000 классов изображений
- ▶ Более 1 000 000 изображений

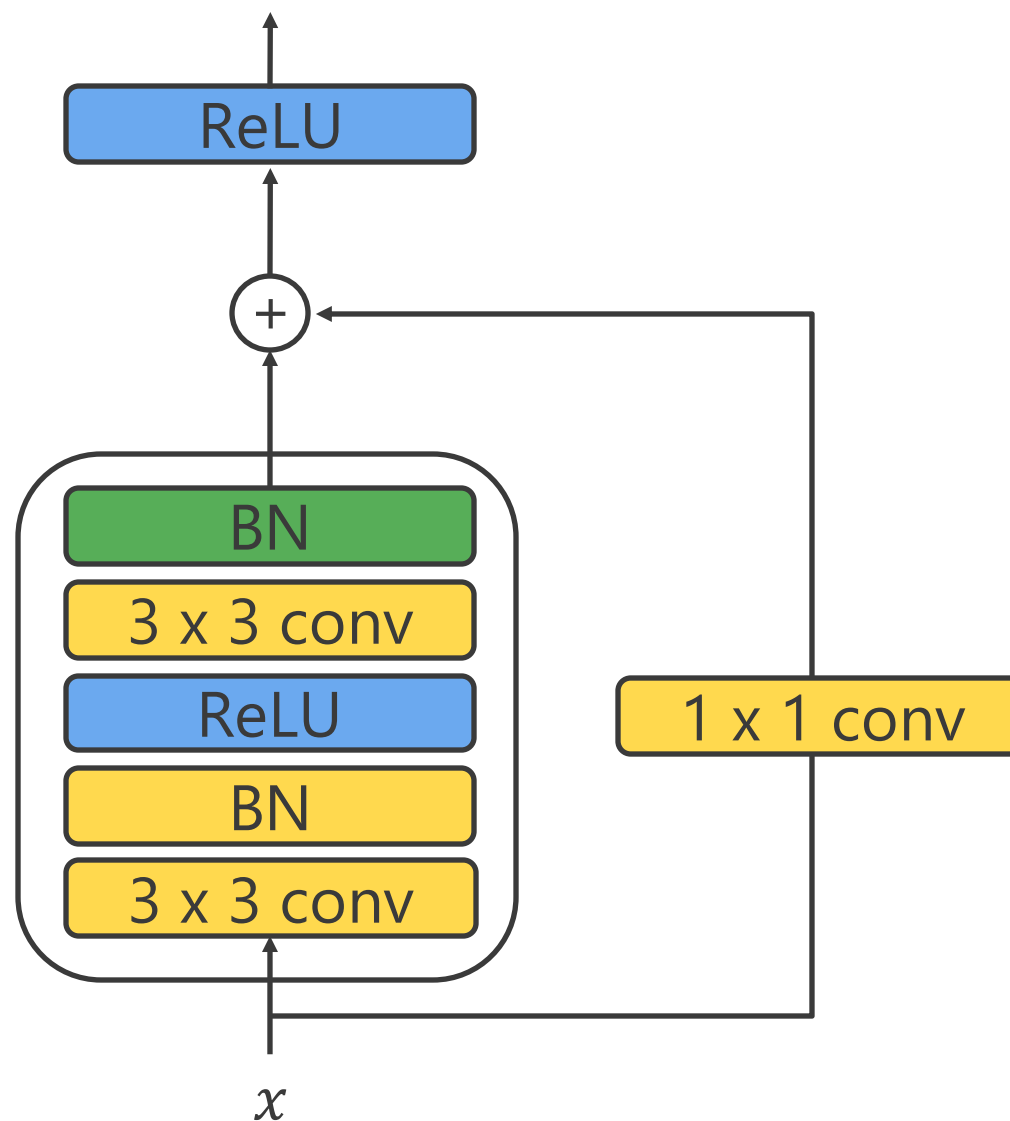
Свертка 1 x 1



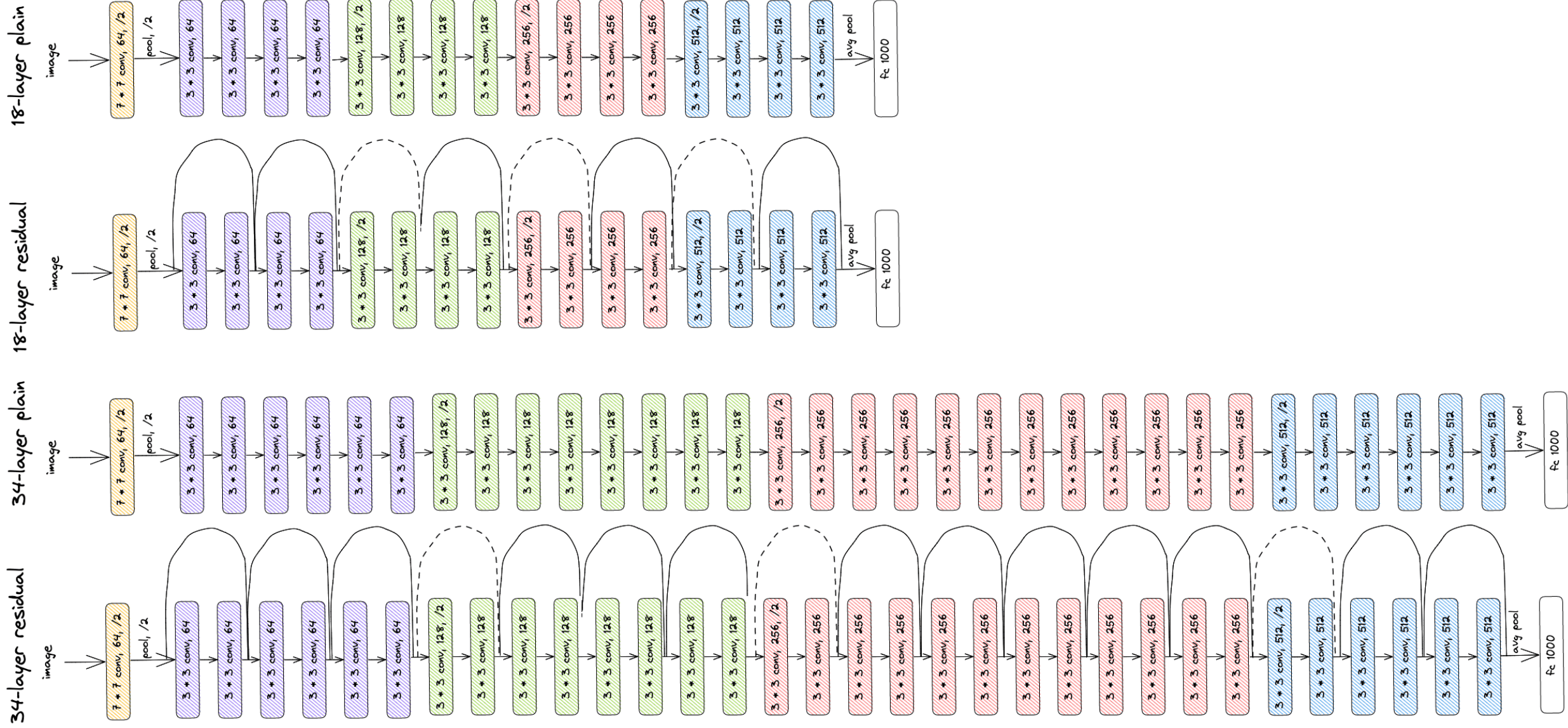
GoogLeNet



Остаточный (residual) блок



ResNet 18 и 34



The background features a series of overlapping, wavy lines in shades of red, orange, and green, creating a sense of depth and movement. A bright, glowing light source is positioned in the upper center, casting a soft, diffused light across the scene. The overall color palette is warm and vibrant, with a gradient from deep reds to bright yellows and oranges.

Перенос обучения (transfer learning)

ImageNet



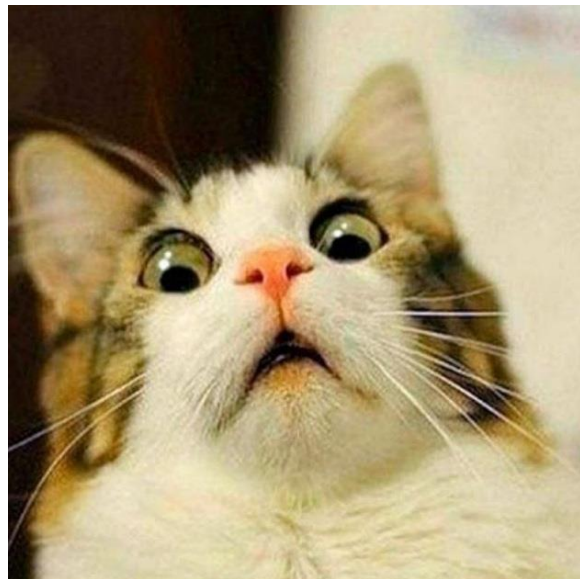
Подробнее: <https://image-net.org/challenges/LSVRC>

- ▶ ImageNet Large Scale Visual Recognition Challenge (ILSVRC)
- ▶ 1000 классов изображений
- ▶ Более 1 000 000 изображений

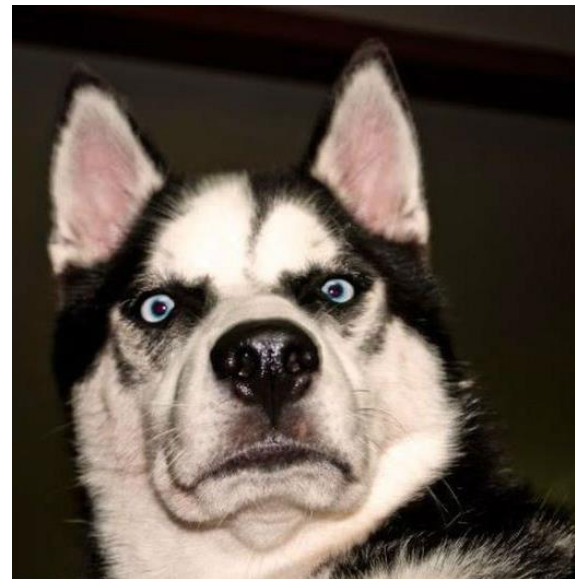
ImageNet

- ▶ Много обученных моделей
- ▶ Тысячи часов на GPU для обучения
- ▶ Можно ли переиспользовать обученные модели в других задачах?

Задача

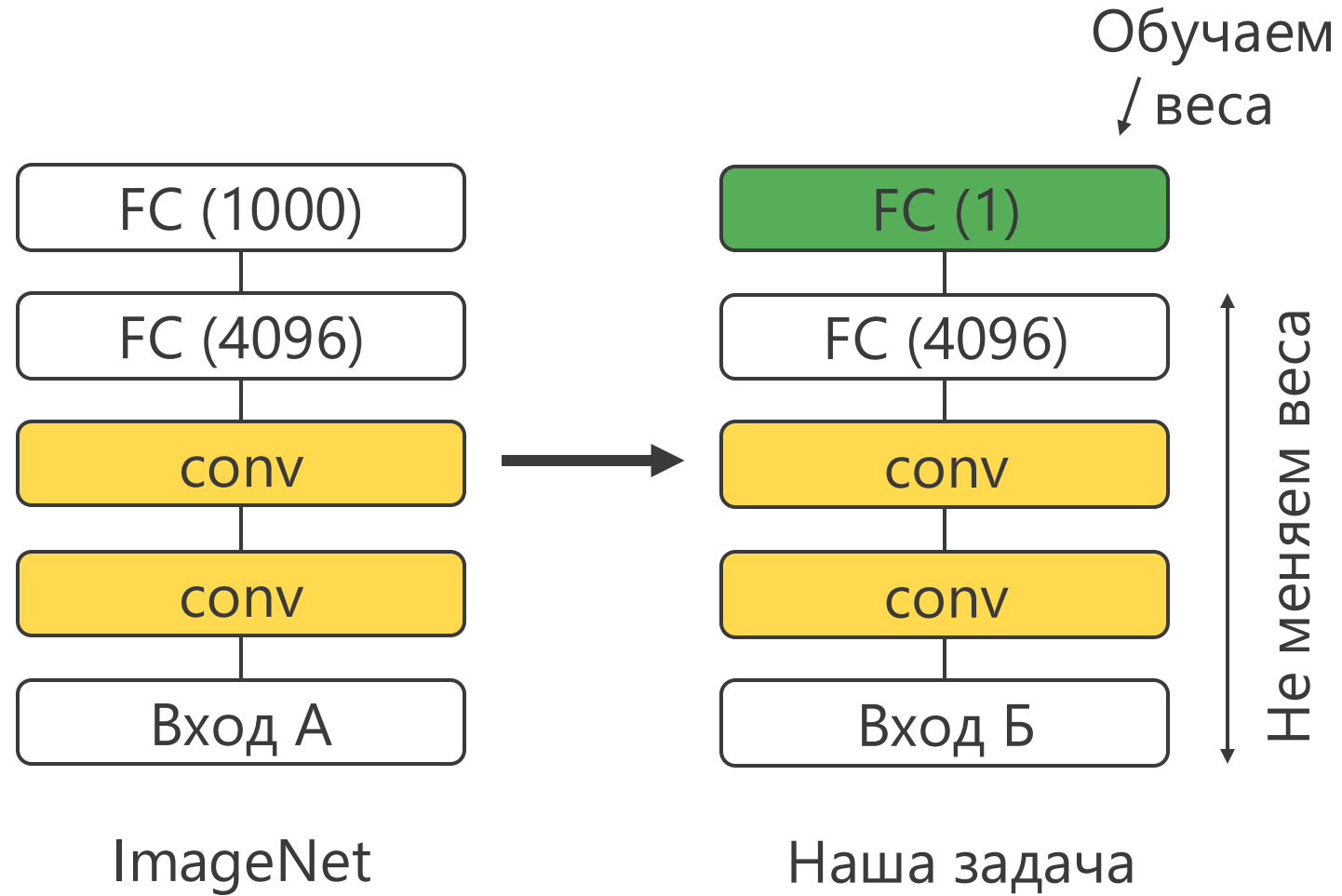


VS



- ▶ Рассмотрим произвольную задачу классификации изображений
- ▶ Как использовать модели, обученные на ImageNet?

Дообучение

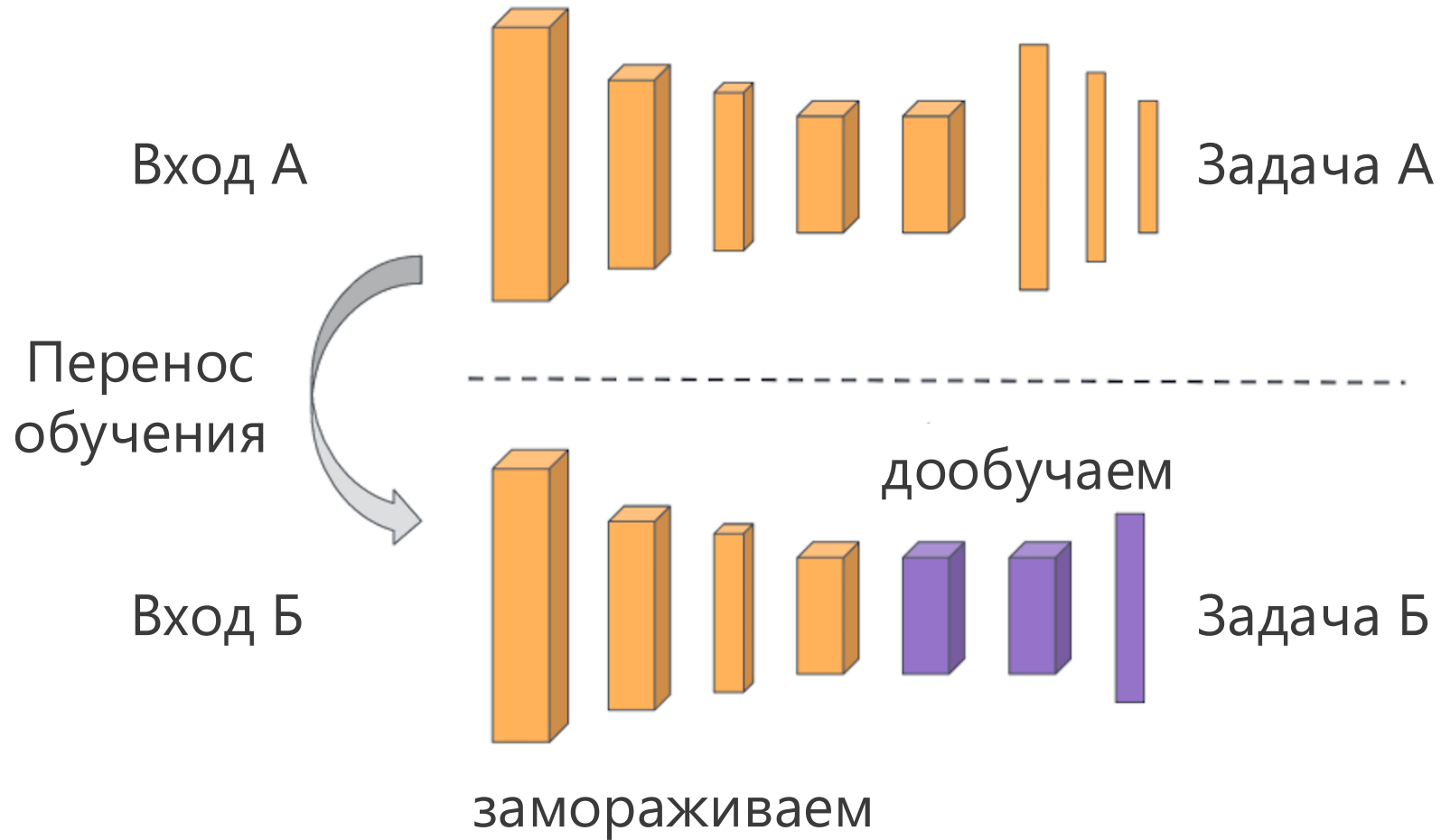


Дообучение

Если данных мало:

- ▶ Берем обученную модель из другой задачи (ImageNet)
- ▶ Заменяем выходной слой на один слой с нужным числом выходов
- ▶ Обучаем на своих данных только новый слой
- ▶ Остальные слои модели замораживаем (не обучаем)

Перенос обучения (fine-tuning)



Дообучение

Если данных достаточно много:

- ▶ Берем обученную модель из другой задачи (ImageNet)
- ▶ Заменяем несколько последних слоев на **несколько** новых
- ▶ Обучаем только новые слои
- ▶ Остальные слои модели замораживаем

Дообучение

- ▶ Остальные слои обученной модели можно не замораживать
- ▶ Их можно дообучить на новых данных
- ▶ Но с меньшим шагом обучения (learning rate)

Перенос обучения

Когда перенос обучения работает:

- ▶ Когда задача A (ImageNet) похожа на нашу задачу B
- ▶ Одинаковые входы модели
- ▶ Для задачи A много больше данных, чем для задачи B



Распознавание лиц

Задача



Anchor



Positive



Anchor



Negative

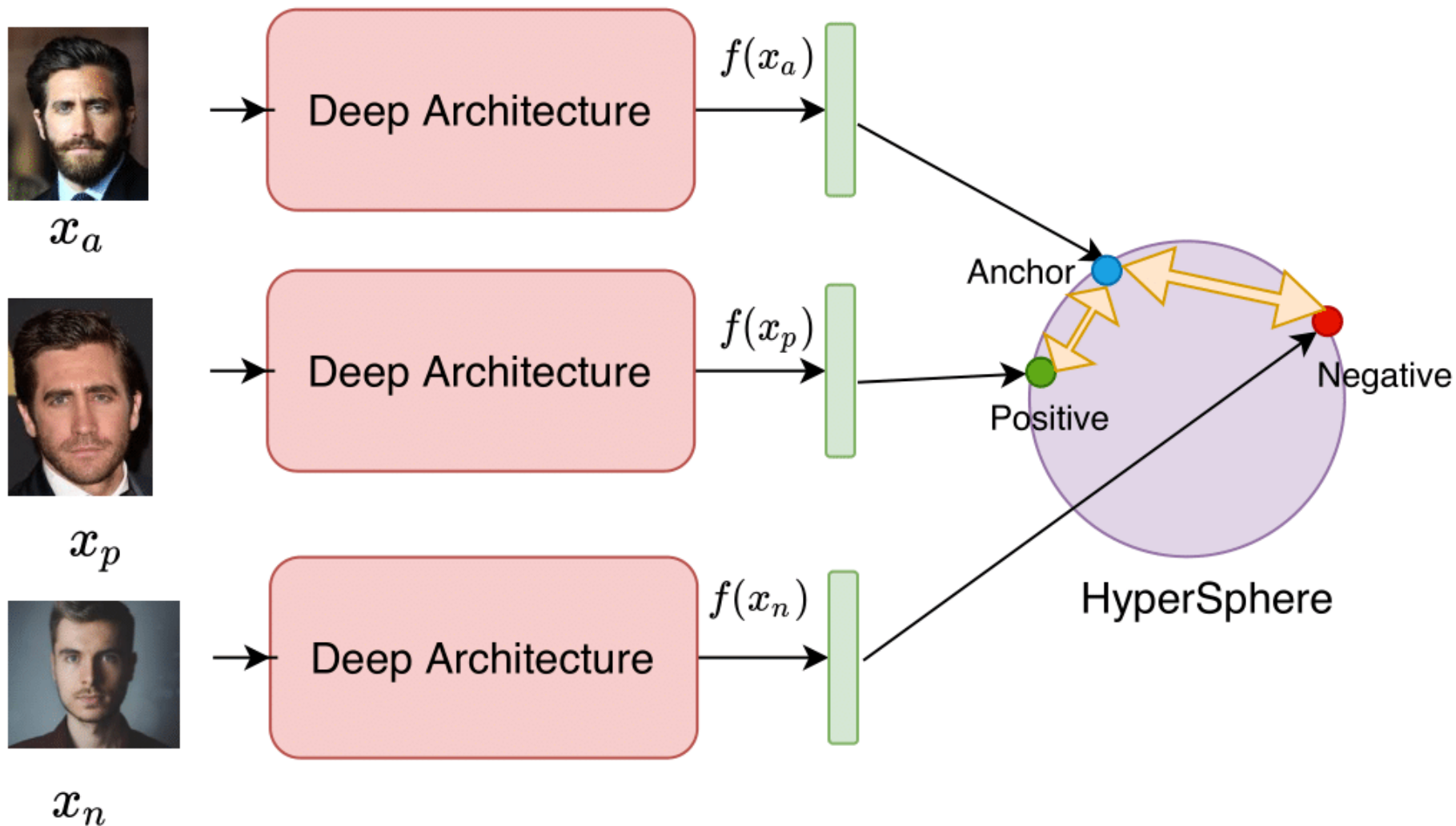
- ▶ Дан набор фотографий студентов вашей группы 😊
- ▶ Нужно обучить нейронную сеть распознавать студентов вашей группы по фото

Распознавание лиц

- ▶ Нейронная сеть должна распознавать вас на любой вашей фото
- ▶ Она также должна определить **любого** студента НЕ вашей группы
- ▶ Для обучения есть только фото студентов вашей группы

- ▶ Как будете действовать? 😊

Архитектура нейронной сети



Источник: <https://pyimagesearch.com/2023/03/06/triplet-loss-with-keras-and-tensorflow/>

Архитектура нейронной сети

- ▶ За основу берем любую (предобученную) нейронную сеть
- ▶ Создаем триплеты фотографий
- ▶ Для каждой фото получаем вектор в скрытом представлении (эмбеddинг) $f(x)$
- ▶ Хотим, чтобы расстояние от якоря $f(x_a)$ до положительного примера $f(x_p)$, было меньше, чем до негативного $f(x_n)$

Triplet loss

Для обучения сети используем triplet loss:

$$L_{triplet} = \text{Max} \left(\|f(x_a) - f(x_p)\|_2^2 - \|f(x_a) - f(x_n)\|_2^2 + \alpha, 0 \right) \rightarrow \min_f$$

α – константа, гиперпараметр. Минимальное расстояние между якорем и негативным примером.

Triplet loss

Обозначим:

$$\text{apDistance} = \|f(x_a) - f(x_p)\|_2^2$$

$$\text{anDistance} = \|f(x_a) - f(x_n)\|_2^2$$

Тогда

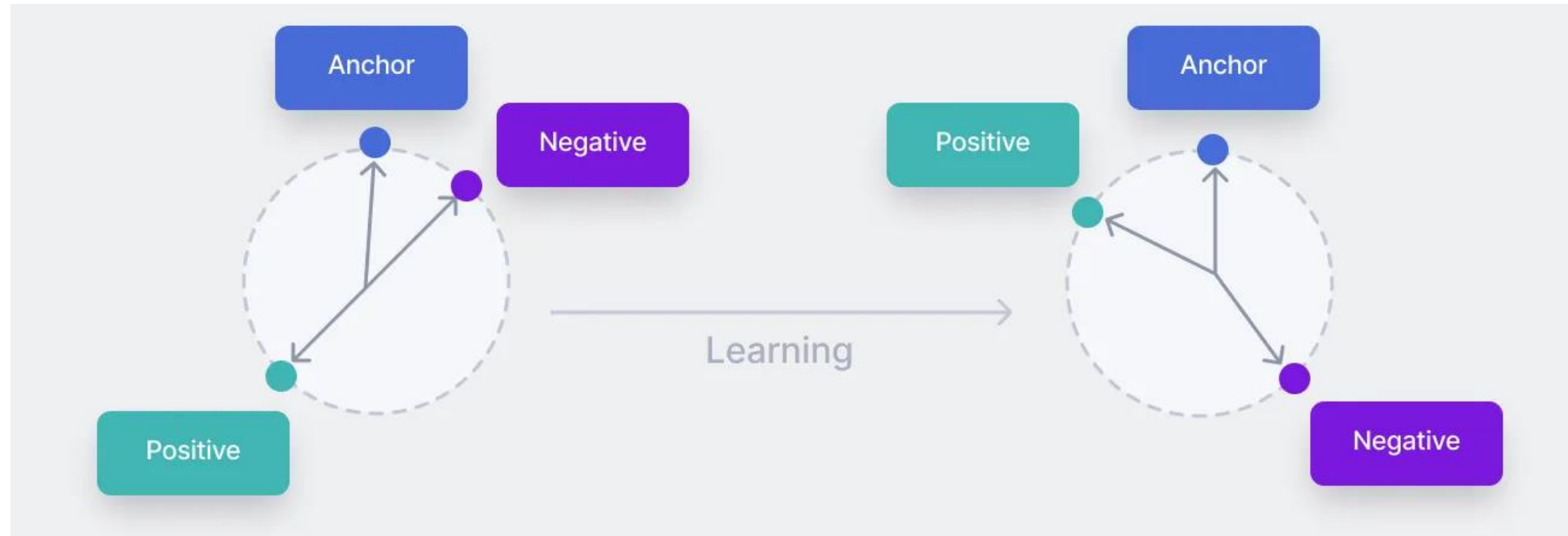
$$L_{\text{triplet}} = \text{Max}(\text{apDistance} - \text{anDistance} + \alpha, 0) \rightarrow \min_f$$

Минимум достигается когда $L_{\text{triplet}} = 0$:

$$\text{apDistance} - \text{anDistance} + \alpha \leq 0$$

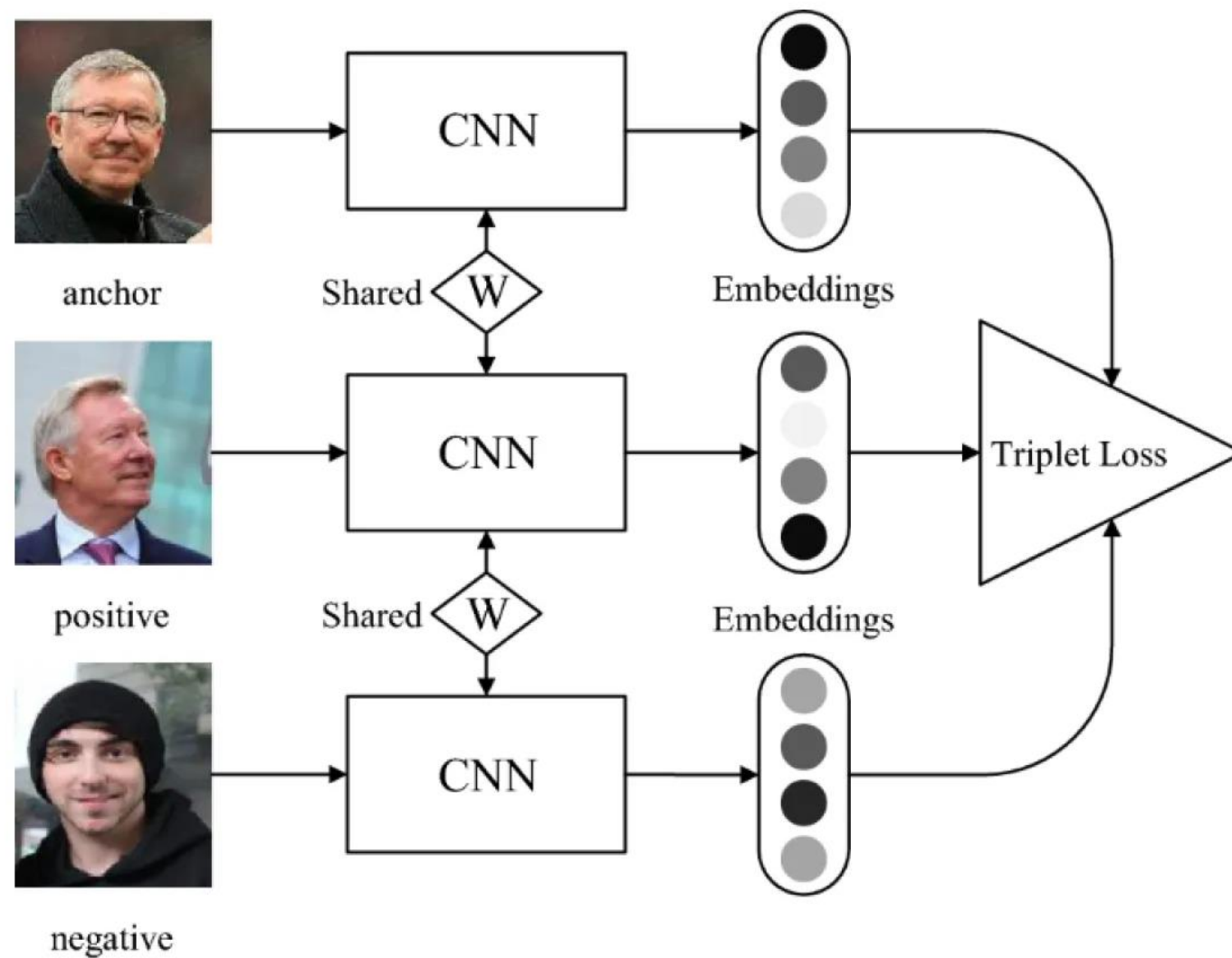
$$\text{anDistance} \geq \text{apDistance} + \alpha$$

Triplet loss



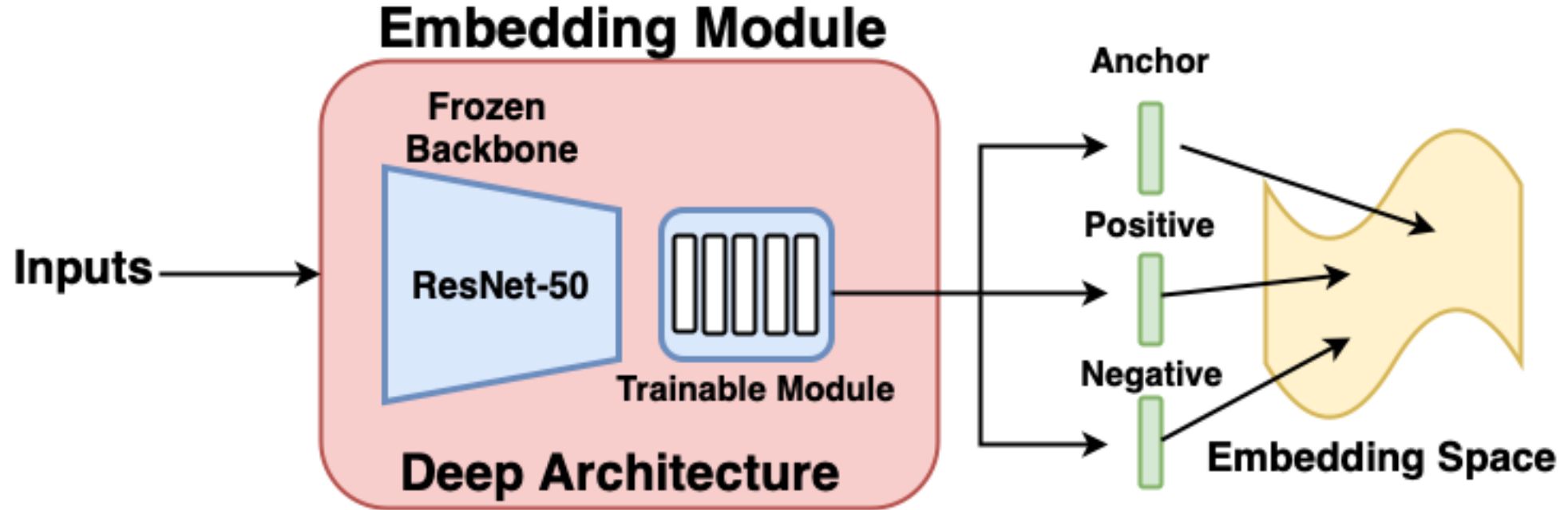
Источник: <https://www.v7labs.com/blog/triplet-loss>

Обучение сети



Обучение сети

Источник: <https://pyimagesearch.com/2023/03/06/triplet-loss-with-keras-and-tensorflow/>



- ▶ Используем перенос обучения
- ▶ За основу берем обученную сеть на ImageNet. Например, ResNet-50)

Какие триплеты бывают?

- ▶ Hard negatives
 - это самые близкие к якорному образцу отрицательные образцы. Они сложны для распознавания моделью и являются самыми информативными для обучения.
- ▶ Semi-hard negatives
 - находятся дальше от якорного образца, чем положительный образец, но всё ещё имеют положительное значение потерь.
- ▶ Easy negatives
 - расположены дальше всего от якорного образца. Они слишком просты для распознавания моделью и не предоставляют полезной информации для обучения.

Приложения трипленной функции потерь

- ▶ Распознавание лиц
- ▶ Рекомендательные системы
 - В рекомендательных системах триплетная потеря помогает улучшить ранжирование рекомендаций. Якорь — это интересующий пользователя товар или контент, положительные примеры — товары, похожие на предпочитаемые пользователем, а отрицательные — менее релевантные предложения. Система учится выделять именно те элементы, которые действительно интересны конкретному пользователю.
- ▶ Поиск изображений по содержанию
 - Использование триплетной функции потерь позволяет обучать модели поиска изображений по содержанию. Здесь якорем является искомое изображение, положительные примеры — изображения схожей тематики, а отрицательные — совершенно разные по смыслу изображения. Это помогает повысить точность поиска по визуальным признакам.
- ▶ Кластеризация объектов
- ▶ Обнаружение подделок, аномалий и контроль качества
 - В приложениях по проверке подлинности документов или товаров триплетная функция помогает определить разницу между оригинальными и поддельными экземплярами. Якорь — оригинал, положительные примеры — оригинальные образцы, а отрицательные — подделки.

Поставьте свою оценку лекции

